

SIMATS ENGINEERING

MODEL PRACTICAL EXAMINATION – Aug. 2026

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

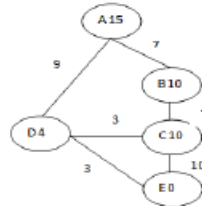
Register Number	Venganapalli Sireesha
Name	192472208

SET:15

1. Write a Python program to implement a Map Colouring problem with the following data. Given a map of countries, and a fixed set of colours {Red, Green, Blue}, assign a colour to each region in the map in such a way that no two adjacent regions have the same colour. Solve the above Map Colouring problem using backtracking in Constraint Satisfaction Procedure.



2. Write a Python program to apply A* search for the given tree, in which 'A' is the initial state and 'E' goal state. For each node in each stage, obtain cost estimates where g(n) is the numeral by the side of an arc and h(n) is the numeral at the node.



3. Write a Prolog program to implement Inference Engine with the following facts
interpret(true) :- !.
interpret((GoalA,GoalB)) :- !,
interpret(GoalA),
interpret(GoalB).
4. Write a Prolog program to find the number of vowels in the following fact.
This is my first Degree in Saveetha School of Engineering.

Laboratory Exam Component - 100 marks			
Programs (4)			
Description	Marks Distribution	Total Marks	Marks Obtained
Aim & Algorithm	2.5*4 = 10 Marks	10	
Programs	10*4 = 30 Marks	40	
Debugging	2.5*4 = 10 Marks	10	
Input & Output	05*4 = 20 Marks	20	
Result	2.5*4 = 10 Marks	10	
GitHub content	10 Marks	10	
Total Lab Exam Marks		100	

1. Aim : To write a Python program to solve the Map Colouring problem using Backtracking in Constraint Satisfaction Procedure, where adjacent regions must not have the same colour.

Algorithm:

1. Define the regions and their adjacent regions.
2. Define the available colours: Red, Green and Blue.
3. Select one region at a time.
4. Assign a colour if it does not conflict with adjacent regions.
5. If no colour is possible, backtrack and change the previous assignment.
6. Continue until all regions are coloured.

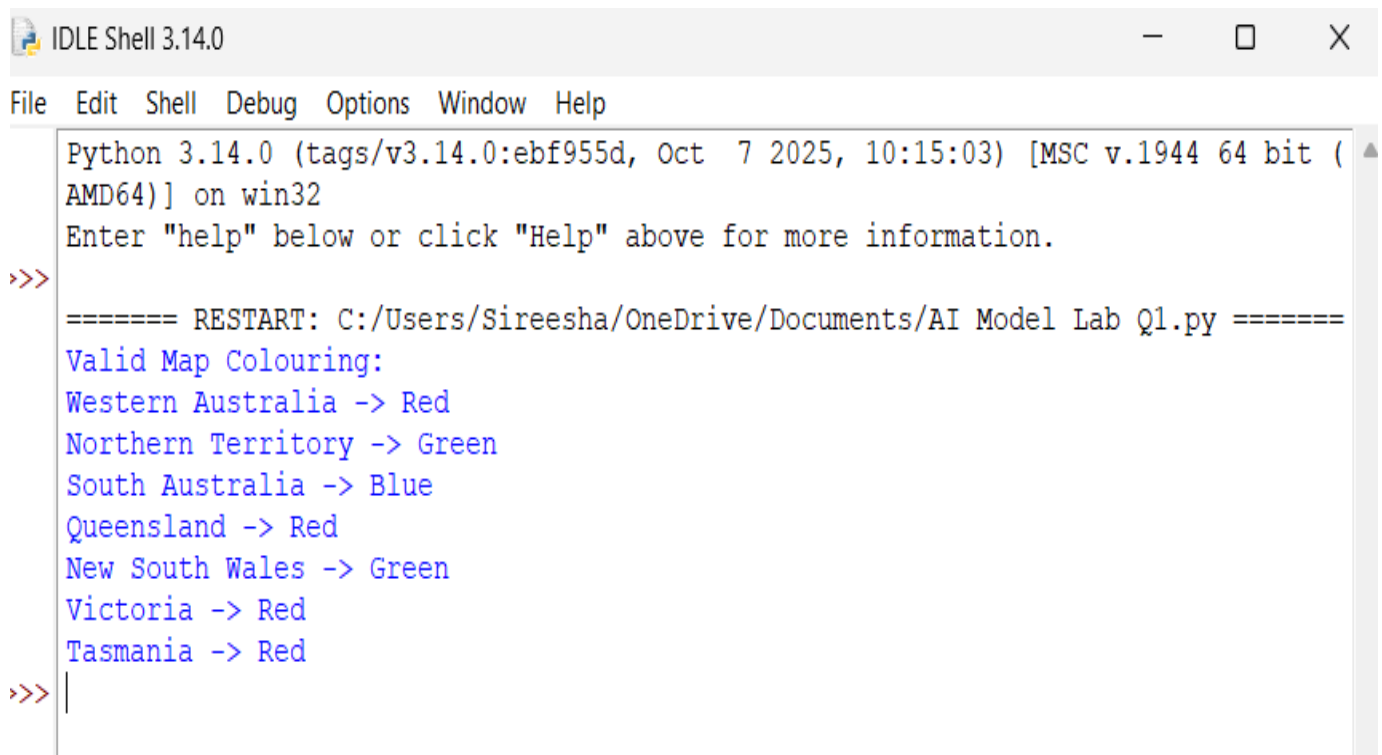
Program

```
*AI Model Lab Q1.py - C:/Users/Sireesha/OneDrive/Documents/AI Model Lab Q1.py (3.14.0)*
File Edit Format Run Options Window Help
regions = [
    "Western Australia",
    "Northern Territory",
    "South Australia",
    "Queensland",
    "New South Wales",
    "Victoria",
    "Tasmania"
]
colors = ["Red", "Green", "Blue"]
neighbors = {
    "Western Australia": ["Northern Territory", "South Australia"],
    "Northern Territory": [
        "Western Australia", "South Australia", "Queensland"
    ],
    "South Australia": [
        "Western Australia", "Northern Territory",
        "Queensland", "New South Wales", "Victoria"
    ],
    "Queensland": [
        "Northern Territory", "South Australia", "New South Wales"
    ],
    "New South Wales": [
        "Queensland", "South Australia", "Victoria"
    ],
    "Victoria": ["South Australia", "New South Wales"],
    "Tasmania": []
}
assignment = {}
def is_safe(region, color):
    for neighbor in neighbors[region]:
        if assignment.get(neighbor) == color:
            return False
    return True
def backtrack(index):
    if index == len(regions):
        return True
    region = regions[index]
    for color in colors:
        if is_safe(region, color):
            assignment[region] = color
            if backtrack(index + 1):
                return True
            del assignment[region]
    return False

if backtrack(0):
    print("Valid Map Colouring:")
    for region in regions:
        print(region, "->", assignment[region])
else:
    print("No valid colouring found.")
```

Ln: 52 Col: 3

Output



```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
==== RESTART: C:/Users/Sireesha/OneDrive/Documents/AI Model Lab Q1.py =====
Valid Map Colouring:
Western Australia -> Red
Northern Territory -> Green
South Australia -> Blue
Queensland -> Red
New South Wales -> Green
Victoria -> Red
Tasmania -> Red
>>> |
```

Result : Thus, the Map Colouring problem was successfully solved using Backtracking in Constraint Satisfaction.

2.Aim: To write a Python program to implement A* Search for finding the optimal path from the initial node A15 to the goal node E0.

Algorithm

1. Start from the initial node A15.
2. Set $g(n) = 0$ for the initial node.
3. Calculate $f(n) = g(n) + h(n)$.
4. Select the node having the smallest $f(n)$.
5. Expand the selected node and calculate costs for its neighbours.
6. Continue until the goal node E0 is reached.
7. Display the optimal path and total cost.

Program

```
*untitled*
File Edit Format Run Options Window Help
import heapq
graph = {
    "A15": [("D4", 9), ("B10", 7)],
    "D4": [("A15", 9), ("C10", 3), ("E0", 3)],
    "B10": [("A15", 7), ("C10", 7)],
    "C10": [("B10", 7), ("D4", 3), ("E0", 10)],
    "E0": [("D4", 3), ("C10", 10)]
}
heuristic = {
    "A15": 15,
    "B10": 10,
    "C10": 10,
    "D4": 4,
    "E0": 0
}
def a_star(start, goal):
    open_list = []
    heapq.heappush(open_list, (heuristic[start], 0, start))
    g_cost = {start: 0}
    parent = {start: None}
    while open_list:
        f, current_g, current = heapq.heappop(open_list)
        if current == goal:
            path = []
            while current is not None:
                path.append(current)
                current = parent[current]
            path.reverse()
            return path, g_cost[goal]
        for neighbor, cost in graph[current]:
            new_g = current_g + cost
            if neighbor not in g_cost or new_g < g_cost[neighbor]:
                g_cost[neighbor] = new_g
                f_cost = new_g + heuristic[neighbor]
                parent[neighbor] = current
                heapq.heappush(
                    open_list,
                    (f_cost, new_g, neighbor)
                )
    return None, None
path, cost = a_star("A15", "E0")
print("Optimal Path:", " -> ".join(path))
print("Total Cost:", cost)
```

Ln: 43 Col: 26

Output

```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> ===== RESTART: C:/Users/Sireesha/OneDrive/Documents/AI Model Lab Q2.py =====
Optimal Path: A15 -> D4 -> E0
Total Cost: 12
>>> |
```

Result : Thus, the A* Search algorithm successfully finds the optimal path : A15 → D4 → E0 with a minimum cost of 12.

3. Aim : To write a Prolog program to implement a simple Inference Engine using facts and rules for true and conjunction of goals.

Algorithm

1. Define the interpret(true) rule.
2. Define the rule for interpreting two goals joined by a conjunction.
3. Interpret the first goal.
4. Interpret the second goal.
5. Execute the required goals and display the result.

Program

```
File Edit View

% Simple Inference Engine

student(sireesha).
course(ai).

interpret(true) :- !.

interpret((GoalA, GoalB)) :-
    !,
    interpret(GoalA),
    interpret(GoalB).

interpret(Goal) :-
    call(Goal).
```

Output

```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/Sireesha/OneDrive/Documents/AI Prolog Experiments/inference.pl').
compiling C:/Users/Sireesha/OneDrive/Documents/AI Prolog Experiments/inference.pl for byte code...
C:/Users/Sireesha/OneDrive/Documents/AI Prolog Experiments/inference.pl compiled, 13 lines read - 1007 bytes written
(32 ms) yes
| ?- interpret(student(sireesha)).

yes
| ?- interpret(course(ai)).

yes
| ?- interpret((student(sireesha), course(ai))).

yes
| ?-
```

Result : Thus, the Prolog program successfully implements a simple Inference Engine for evaluating individual and conjunctive goals.

4. Aim : To write a Prolog program to find the number of vowels in the given sentence.

Algorithm

1. Store the given sentence as a list of characters.
2. Check each character.
3. If the character is a, e, i, o, u, increase the count.
4. Ignore consonants, spaces and punctuation marks.
5. Continue until all characters are checked.
6. Display the total number of vowels.

Program

```
File Edit View

% Program to count the number of vowels

vowel(a).
vowel(e).
vowel(i).
vowel(o).
vowel(u).

vowel('A').
vowel('E').
vowel('I').
vowel('O').
vowel('U').

count_vowels([], 0).

count_vowels([H|T], Count) :-
    vowel(H),
    count_vowels(T, Rest),
    Count is Rest + 1.

count_vowels([_|T], Count) :-
    \+ vowel(_),
    count_vowels(T, Count).

count_sentence_vowels(String, Count) :-
    string_chars(String, Characters),
    count_vowels(Characters, Count).
```

Output

Number of vowels = 19

```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/Sireesha/OneDrive/Documents/AI Prolog Experiments/number of vowels.pl').
compiling C:/Users/Sireesha/OneDrive/Documents/AI Prolog Experiments/number of vowels.pl for byte code...
C:/Users/Sireesha/OneDrive/Documents/AI Prolog Experiments/number of vowels.pl compiled, 27 lines read - 2130 by

yes
| ?- count_sentence_vowels(
'This is my first Degree in Saveetha School of Engineering.',
Count).

Count = 19 ?

yes
| ?- |
```

Result : Thus, the Prolog program successfully counts the vowels in the given sentence.